

# ENGG1910: Demystifying AI

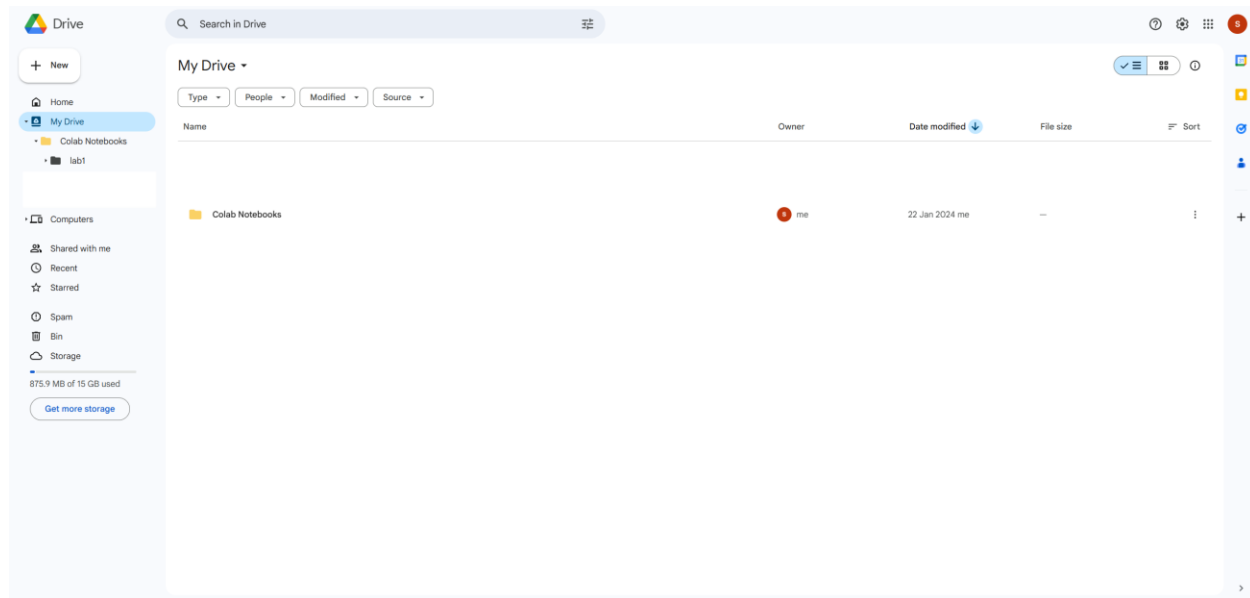
## Lab 1 Guidelines

### 1 Objectives

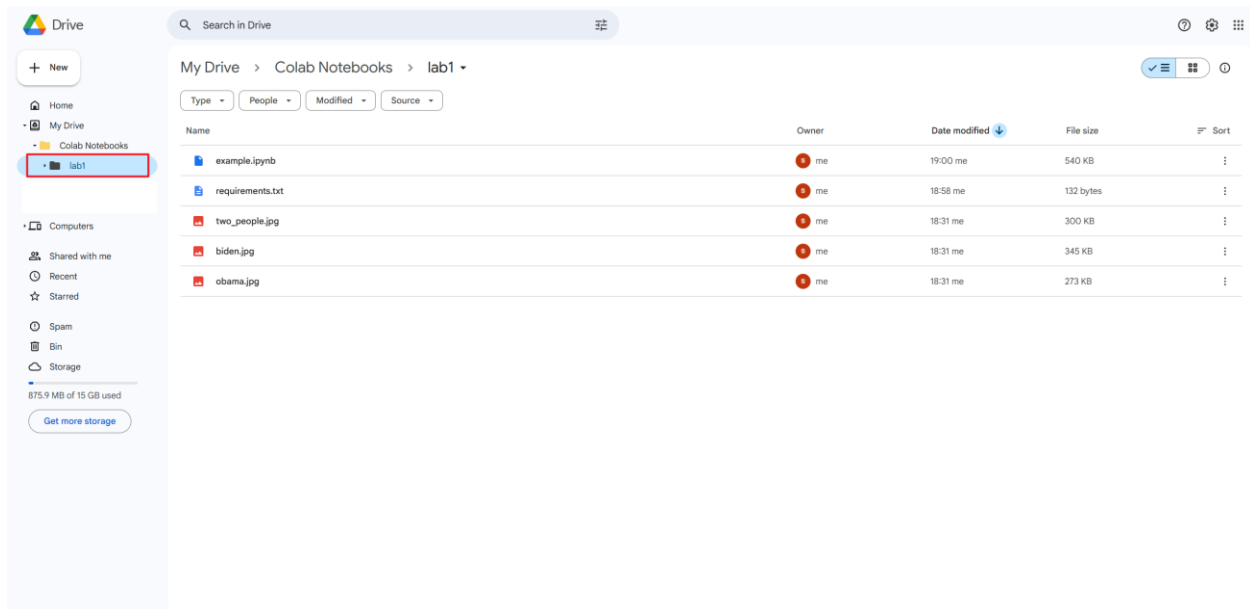
- 1.1 Try to use an open-source project on your PC.
- 1.2 In this lab, students are expected to learn the basic procedure of **face recognition, including train data and feature vector**.
- 1.3 Try to make modifications based on open source projects to achieve specific face recognition.

### 2 Detailed Steps

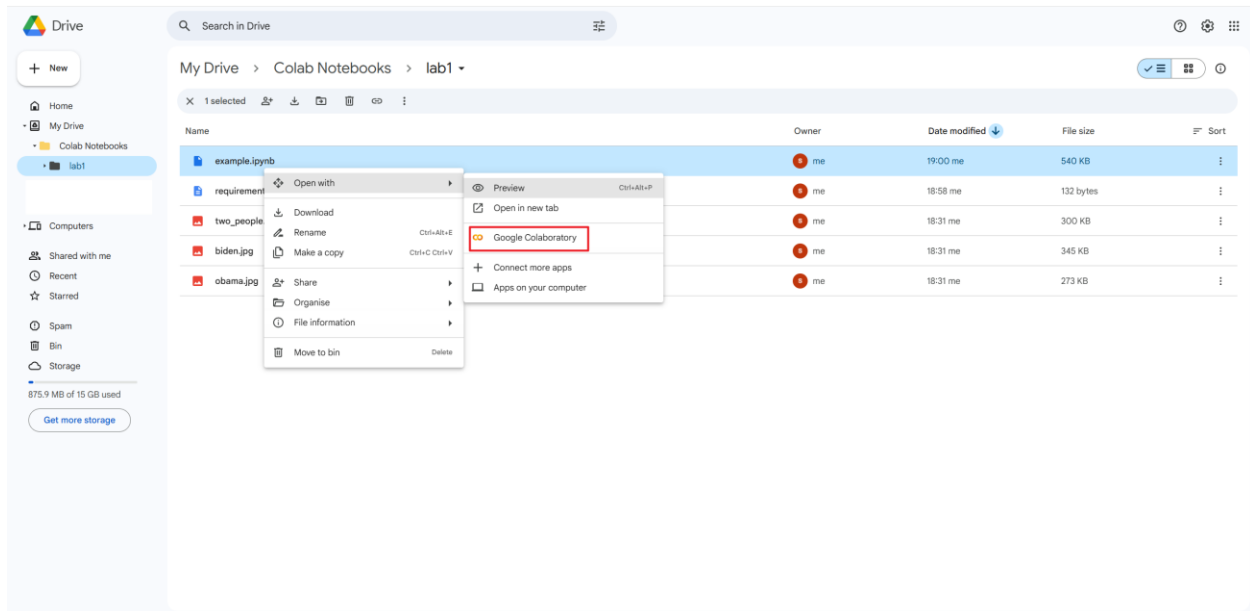
- 2.1 Prepare the workplace on Google Colab.
  - 2.1.1 Log in to Google Drive using your Google account. Unzip the lab file and upload the “Colab Notebooks” folder to the **root directory** of your Google Drive.



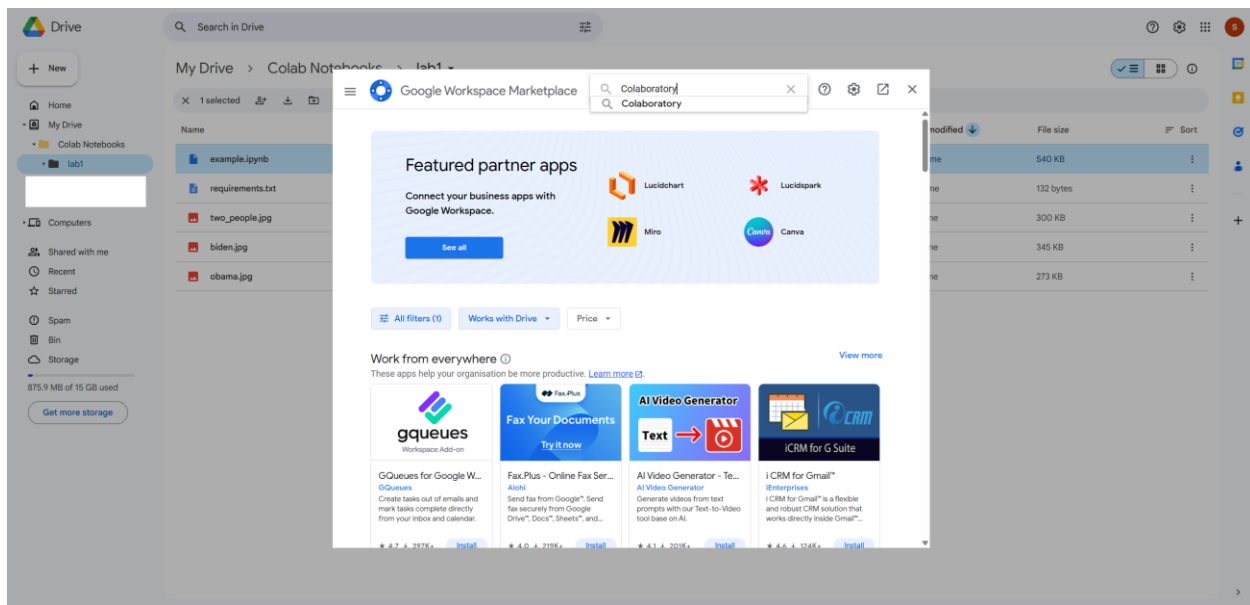
- 2.1.2 Open the “Colab Notebooks” folder and open the “lab1” folder inside it. You should see “example.ipynb” “requirements.txt” “two\_people.jpg” “biden.jpg” “Obama.jpg”



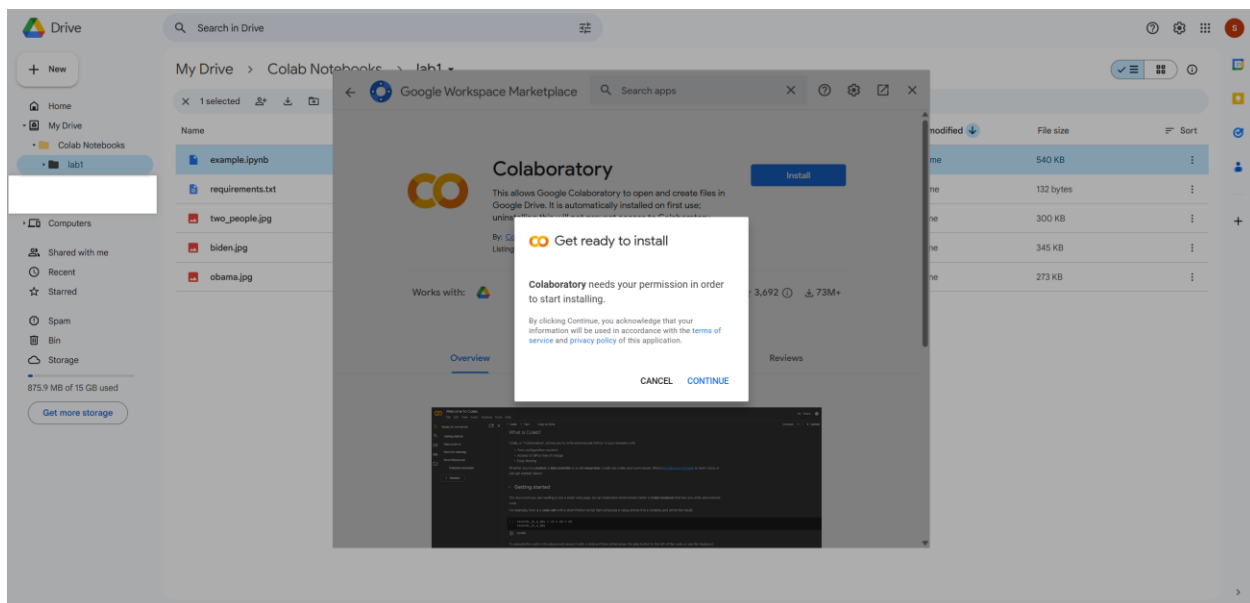
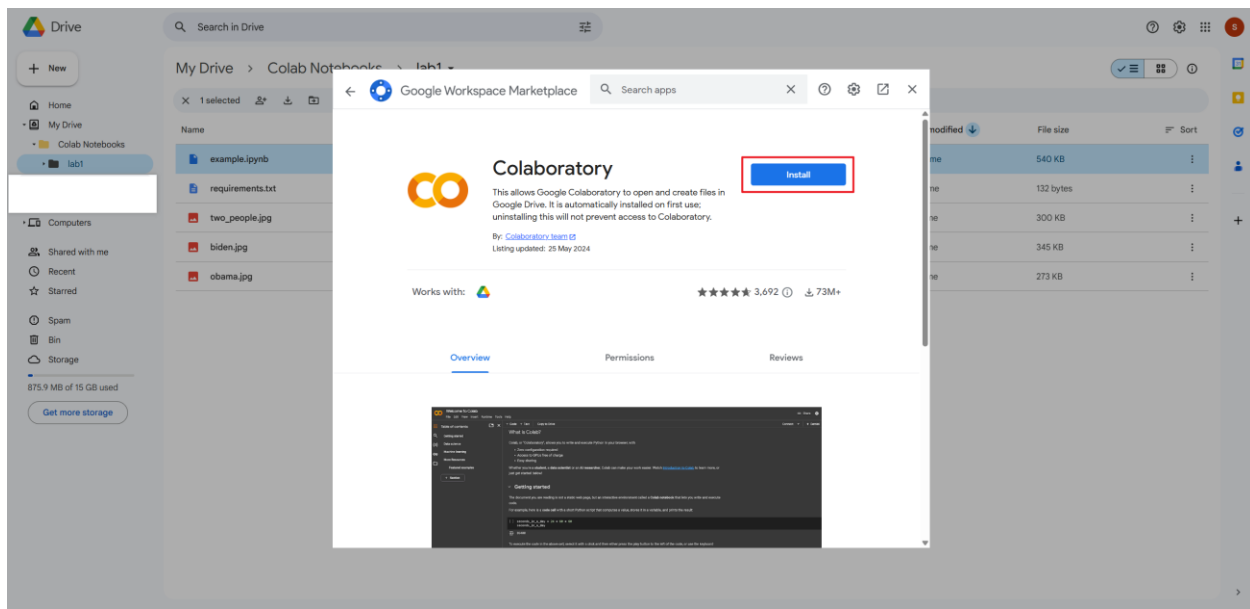
*If you have already seen that option, you can select “Google Colaboratory” directly and proceed to Step 2.2.*



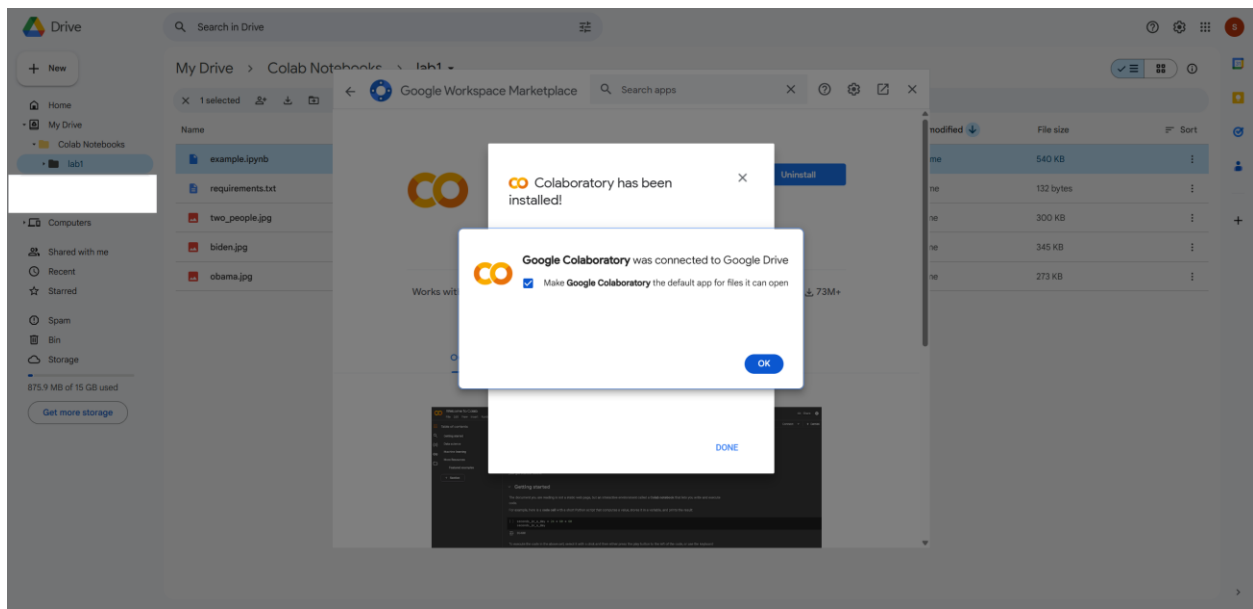
After clicking “Connect more apps”, type “Colaboratory” in the search box to search for the tool, or click on it in the prompt below the search box.



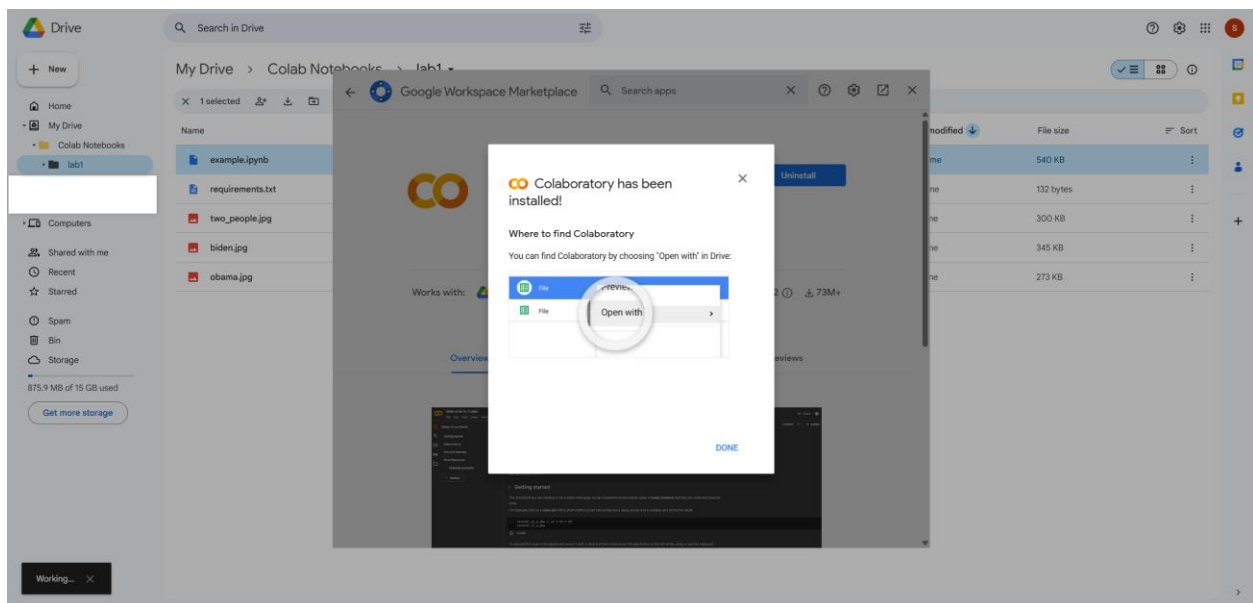
Click on the first option in the search results, then install Google Colaboratory.

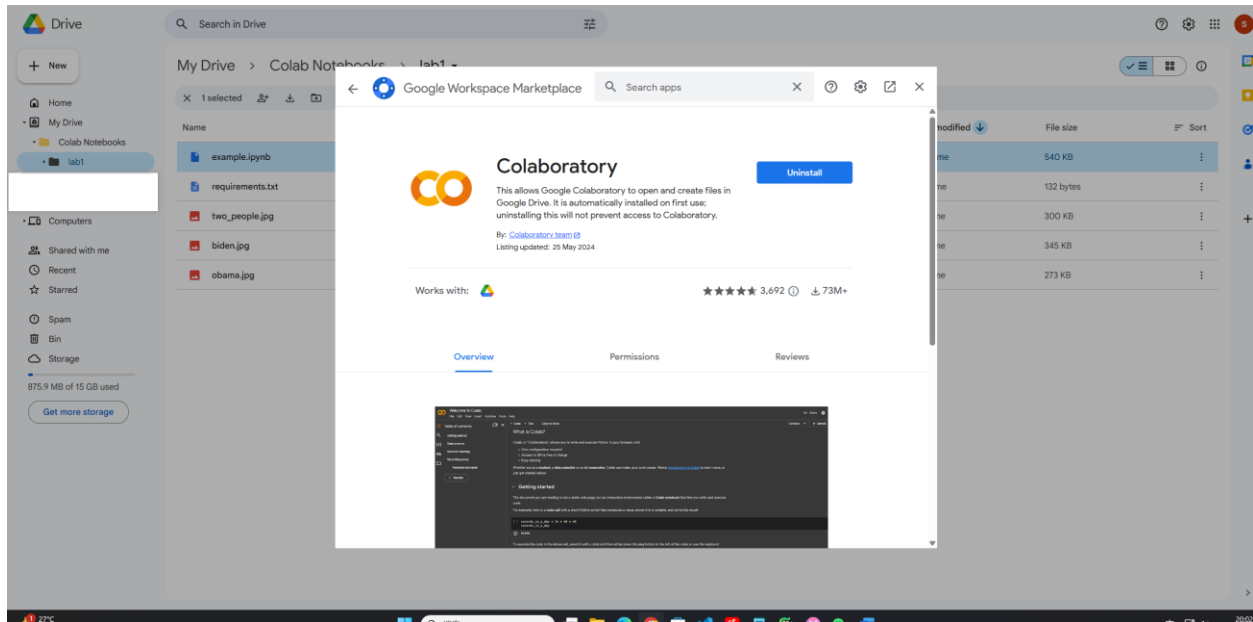


You may need to log in to your Google account again and give permission to install the tool.

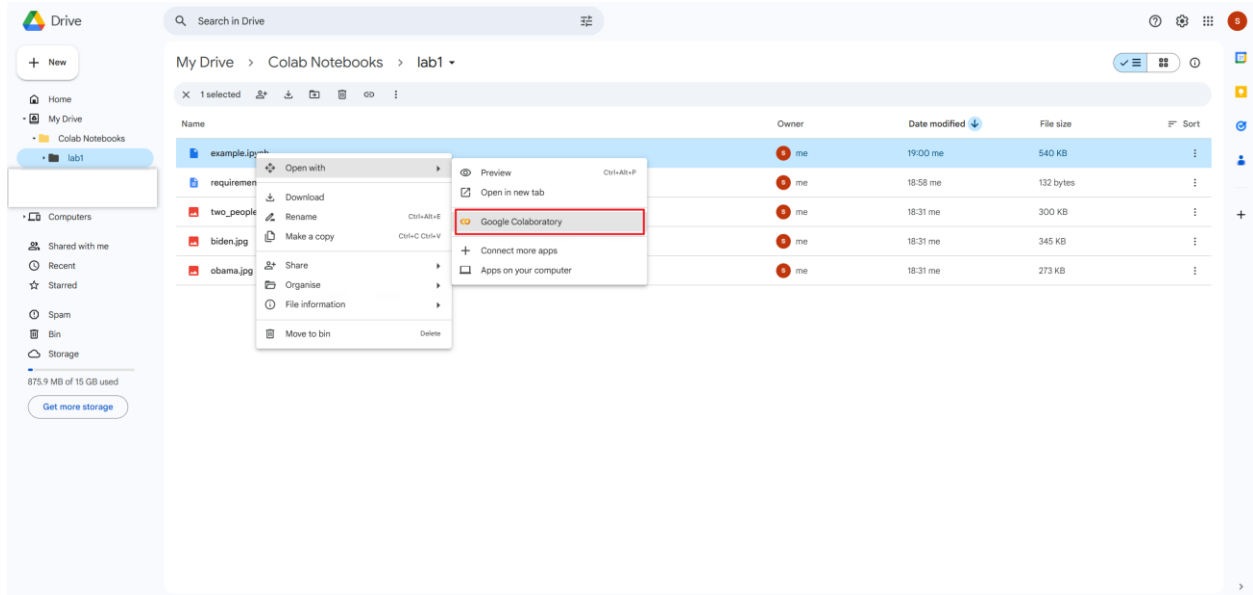


After installing Colaboratory, you can make it the default app for opening “ipynb” files if you wish.





Close the windows and go back to the “lab1” folder. Right-click the “example.ipynb” file again and select “Google Colaboratory” to start the lab.



Upload other files in lab1 folder through this button and upload other files

example.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

Upload to session storage

sample\_data

Install the environment

```
[ ] !apt-get update
!apt-get install -y cmake libgtk2.0-dev
!pip install -r requirements.txt
```

Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and run the cell below to see the image.

```
[ ] from PIL import Image, ImageDraw
from IPython.display import display

# The program we will be finding faces on the example below
pil_im = Image.open('two_people.jpg')
display(pil_im)
```

Show hidden output

Learning from example

Now we show the library two different faces (Joe Biden, Barack Obama) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

```
[ ] import face_recognition
import numpy as np
from PIL import Image, ImageDraw, ImageFont
from IPython.display import display

# This is an example of running face recognition on a single image
# and drawing a box around each person that was identified.

# Load a sample picture and learn how to recognize it.
obama_image = face_recognition.load_image_file('obama.jpg')
obama_face_encoding = face_recognition.face_encodings(obama_image)[0]
```

Variables Terminal Python 3

example.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

sample\_data

biden.jpg

obama.jpg

requirements.txt

two\_people.jpg

Install the environment

```
[ ] !apt-get update
!apt-get install -y cmake libgtk2.0-dev
!pip install -r requirements.txt
```

Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and run the cell below to see the image.

```
[ ] from PIL import Image, ImageDraw
from IPython.display import display

# The program we will be finding faces on the example below
pil_im = Image.open('two_people.jpg')
display(pil_im)
```

Show hidden output

Learning from example

Now we show the library two different faces (Joe Biden, Barack Obama) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

```
[ ] import face_recognition
import numpy as np
from PIL import Image, ImageDraw, ImageFont
from IPython.display import display

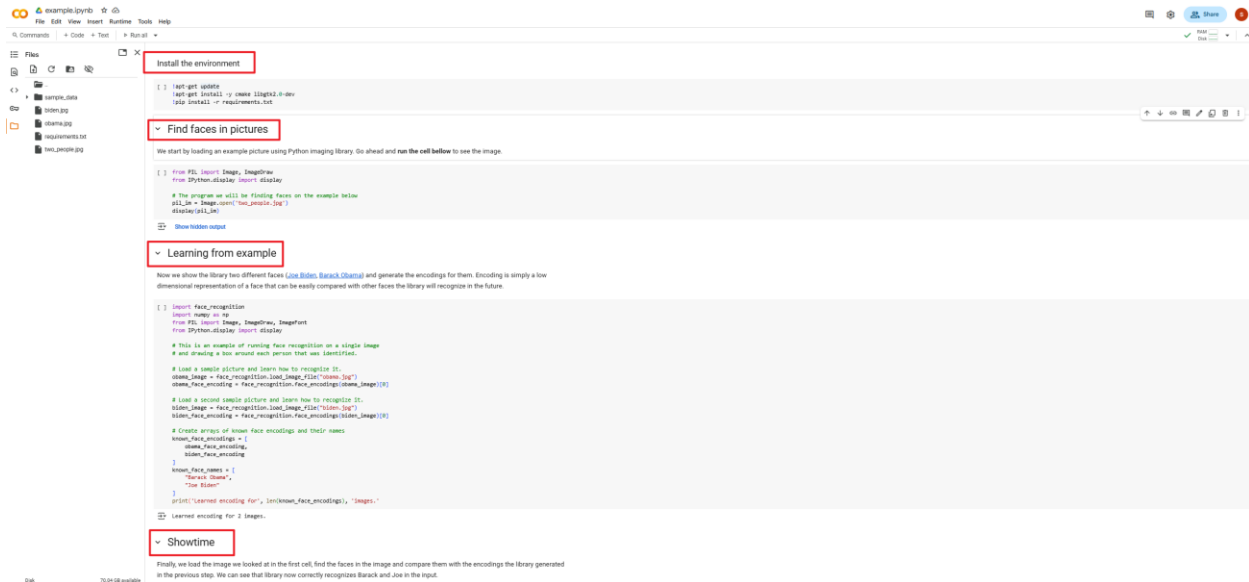
# This is an example of running face recognition on a single image
# and drawing a box around each person that was identified.

# Load a sample picture and learn how to recognize it.
obama_image = face_recognition.load_image_file('obama.jpg')
obama_face_encoding = face_recognition.face_encodings(obama_image)[0]
```

Variables Terminal Python 3

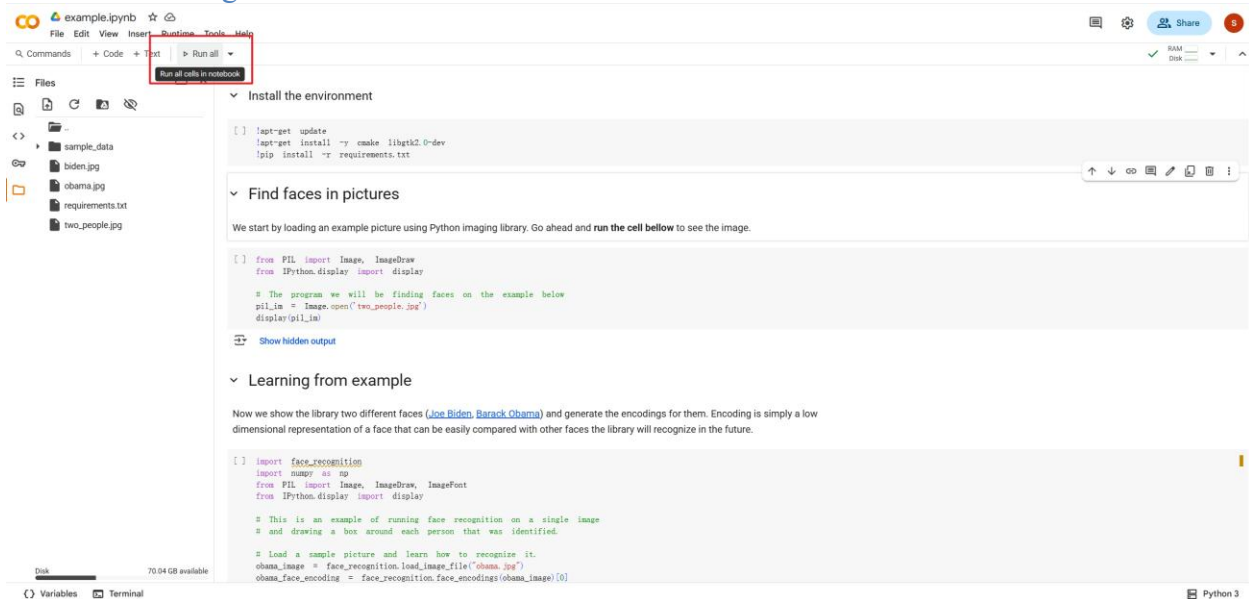
## 2.2 Try the code in Colab

### 2.2.1 Read the code & comments



Read the code and comments, there are four blocks.

### 2.2.2 Run the original demo



### 2.2.3 Results



The original model can accurately pick out Biden and Obama from the images

### 2.2.4 Test the model with different images

- 1) Open the list of documents on the left.
- 2) Select the image name 'biden.jpg' to replace the original image name 'two\_people.jpg'.
- 3) Run the demo again.

example.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

sample\_data

biden.jpg

obama.jpg

requirements.txt

two\_people.jpg

Install the environment

[4] !apt-get update  
!apt-get install -y cmake libgtk2.0-dev  
!pip install -r requirements.txt

Show hidden output

Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and run the cell below to see the image.

[5] from PIL import Image, ImageDraw  
from IPython.display import display  
  
# The program we will be finding faces on the example below  
pil\_im = Image.open('two\_people.jpg')  
display(pil\_im)

Show hidden output

Learning from example

Now we show the library two different faces (Joe Biden, Barack Obama) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

[6] import face\_recognition  
import numpy as np  
from PIL import Image, ImageDraw, ImageFont  
from IPython.display import display  
  
# This is an example of running face recognition on a single image  
# and drawing a box around each person that was identified.  
# Load a sample picture and learn how to recognize it.

Disk 69.42 GB available

Variables Terminal

8:13 PM Python 3

example.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

sample\_data

biden.jpg

obama.jpg

requirements.txt

two\_people.jpg

Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and run the cell below to see the image.

[5] from PIL import Image, ImageDraw  
from IPython.display import display  
  
# The program we will be finding faces on the example below  
pil\_im = Image.open('biden.jpg')  
display(pil\_im)

Show hidden output

Learning from example

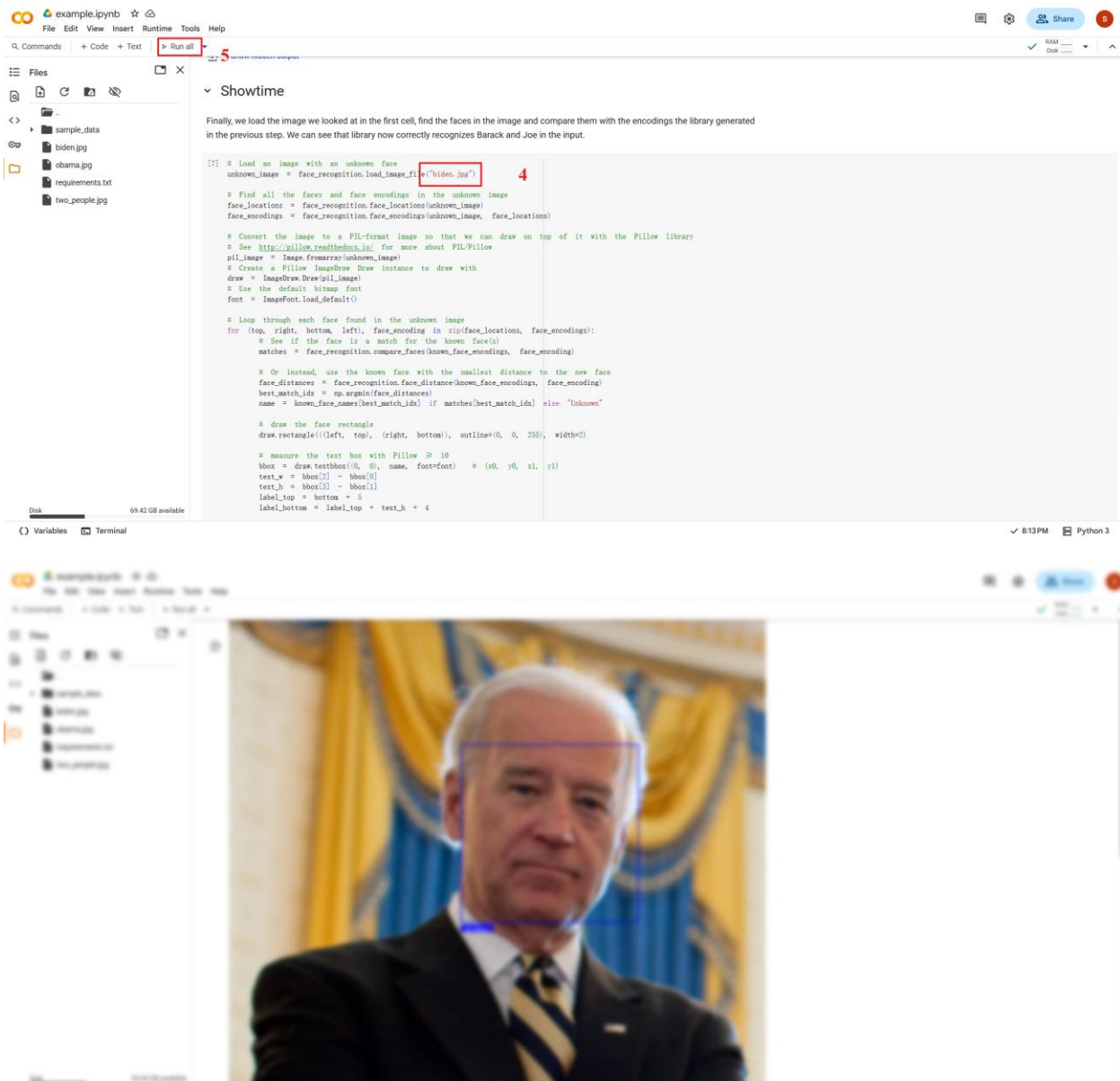
Now we show the library two different faces (Joe Biden, Barack Obama) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

[6] import face\_recognition  
import numpy as np  
from PIL import Image, ImageDraw, ImageFont  
from IPython.display import display  
  
# This is an example of running face recognition on a single image  
# and drawing a box around each person that was identified.  
# Load a sample picture and learn how to recognize it.

Disk 69.42 GB available

Variables Terminal

8:13 PM Python 3



**Need to submit:** The recognized result for “biden.jpg”, named as **Figure 1** in your report.

## 2.3 Try to use your own data

### 2.3.1 Prepare your own face data

We take comedians Xiao Yueyue (xyy) and Guo Degang (gdg) for example



Xiao Yueyue (xxy) and Guo Degang (gdg)



Image to be tested

### 2.3.2 Upload the prepared template to the web page

The screenshot shows a JupyterLab environment. In the file browser on the left, a red box highlights the 'Upload to version storage' button. Below it, a list of files is shown, with 'gdg.png' and 'xxy.png' highlighted in red. The terminal on the right shows the following output:

```
[8] !apt-get update
!apt-get install -y cmake libgtk2.0-dev
!pip install -r requirements.txt

Hit:1 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:2 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64 InRelease
Hit:3 https://security.ubuntu.com/ubuntu jammy-security InRelease
Get:4 https://rfa.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Hit:5 https://archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 https://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 https://archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:8 https://rfa.stat.illinois.edu/ubuntu jammy/main all Packages [9,106 kB]
Hit:9 https://nec.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:10 https://nec.launchpadcontent.net/crashfix/drivers/ppa/ubuntu jammy InRelease
Hit:11 https://nec.launchpadcontent.net/ubuntuwsl/ppa/ubuntu jammy InRelease
Get:12 https://rfa.stat.illinois.edu/ubuntu jammy/main amd64 Packages [2,757 kB]
Fetched 11.9 MB in 2s (7,849 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://rfa.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list entry misspell?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
cmake is already the newest version (3.22.1-1ubuntu1.22.04.2).
libgtk2.0-dev is already the newest version (3.24.33-2ubuntu2.1).
0 upgraded, 0 newly installed, 0 to remove and 42 not upgraded.
Requirement already satisfied: cmake==4.0.3 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 1)) (4.0.3)
Requirement already satisfied: dlib==19.24.6 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 2)) (19.24.6)
Requirement already satisfied: face-recognition==1.3.0 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 3)) (1.3.0)
Requirement already satisfied: face-recognition-models==0.3.0 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 4)) (0.3.0)
Requirement already satisfied: numpy==2.3.1 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 5)) (2.3.1)
Requirement already satisfied: pillow==11.3.0 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 6)) (11.3.0)
Requirement already satisfied: click==8.2.1 in /usr/local/lib/python3.11/dist-packages (from -r requirements.txt (line 7)) (8.2.1)
Requirement already satisfied: dlib==19.7 in /usr/local/lib/python3.11/dist-packages (from face-recognition==1.3.0->-r requirements.txt (line 3)) (19.24.6)

Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and run the cell below to see the image.

from PIL import Image, ImageDraw
```

### 2.3.3 Extra two persons' latent code

Add the relevant code to the Learning From Example block, as follow:

## Learning from example

```
import face_recognition
import numpy as np
from PIL import Image, ImageDraw
from IPython.display import display

# This is an example of running face recognition on a single image
# and drawing a box around each person that was identified.

# Load a sample picture and learn how to recognize it.
obama_image = face_recognition.load_image_file("obama.jpg")
obama_face_encoding = face_recognition.face_encodings(obama_image)[0]

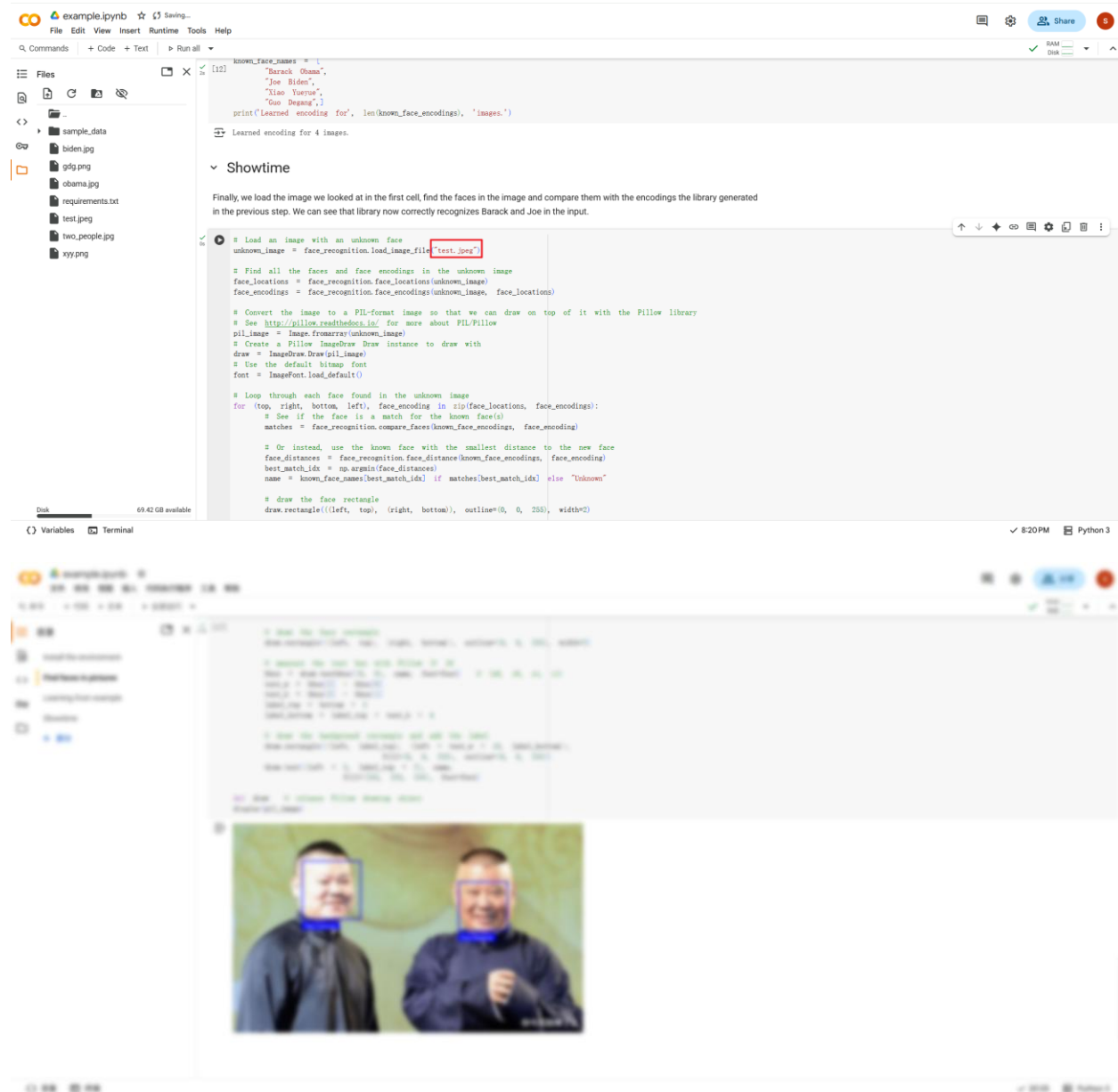
# Load a second sample picture and learn how to recognize it.
biden_image = face_recognition.load_image_file("biden.jpg")
biden_face_encoding = face_recognition.face_encodings(biden_image)[0]

# Load xyy sample picture and learn how to recognize it.
xyy_image = face_recognition.load_image_file("xyy.png")
xyy_face_encoding = face_recognition.face_encodings(xyy_image)[0]

# Load gdg sample picture and learn how to recognize it.
gdg_image = face_recognition.load_image_file("gdg.png")
gdg_face_encoding = face_recognition.face_encodings(gdg_image)[0]

# Create arrays of known face encodings and their names
known_face_encodings = [
    obama_face_encoding,
    biden_face_encoding,
    xyy_face_encoding,
    gdg_face_encoding,
]
known_face_names = [
    "Barack Obama",
    "Joe Biden",
    "Xiao Yueyue",
    "Guo Degang",]
print('Learned encoding for', len(known_face_encodings), 'images.')
```

## 2.3.4 Make a test



**Need to submit:** The screenshot of your codes and recognized result for two people 'test.jpg', named as **Figure 2** and **Figure 3**, respectively in your report.

2.4.1 Test the model with some processed images including color changes, angle changes, face changes, etc.

Follow the procedure above to test with new image.



**Need to submit:** The recognized result, named as **Figure 4** in your report.



**Need to submit:** The recognized result, named as **Figure 5** in your report.



**Need to submit:** The recognized result, named as **Figure 6** in your report.



**Need to submit:** The recognized result, named as **Figure 7** in your report.



**Need to submit:** The recognized result, named as **Figure 8** in your report.

#### [2.5.1 Test the model with unknown people who not in the database](#)

Follow the procedure above to test with unknown people in the cell right below Showtime:

unknown.jpg



**Need to submit:** The recognized result, named as **Figure 9** in your report.

Test another unknown people with the figure: dongkun1.jpg



dongkun1.jpg

**Need to submit:** The recognized result regarding dongkun1.jpg, named as **Figure 10** in your report.

Put the unknown people into the library and generate the encoding for him/her. Follow the steps 2.3.1-2.3.4 and change your codes accordingly.



dongkun1.jpg

Test a picture of an unknown people (e.g., dongkun2.jpg) after you add the face of this unknown people into the library (e.g. dongkun and his corresponding picture: dongkun1.jpg)



dongkun2.jpg

**Need to submit:** The recognized result regarding dongkun1.jpg after we put the face of unknown people into the library (i.e., train the model on the unknown person and let the model recognize this person again by using **another picture** of this person), named as **Figure 11** in your report.